* It combines the direct hardware control & efficiency of low-level langu with the human-readable syntax & portability of high-level lang

History of C Program

* C Programming lang was developed by Dennis Ritchie in 1972 bell laboratories of AT&T located the U.S.A
* he is known as the founder of the C lang
* It was developed to overcome the problem of low level langu like B, BCPL etc.

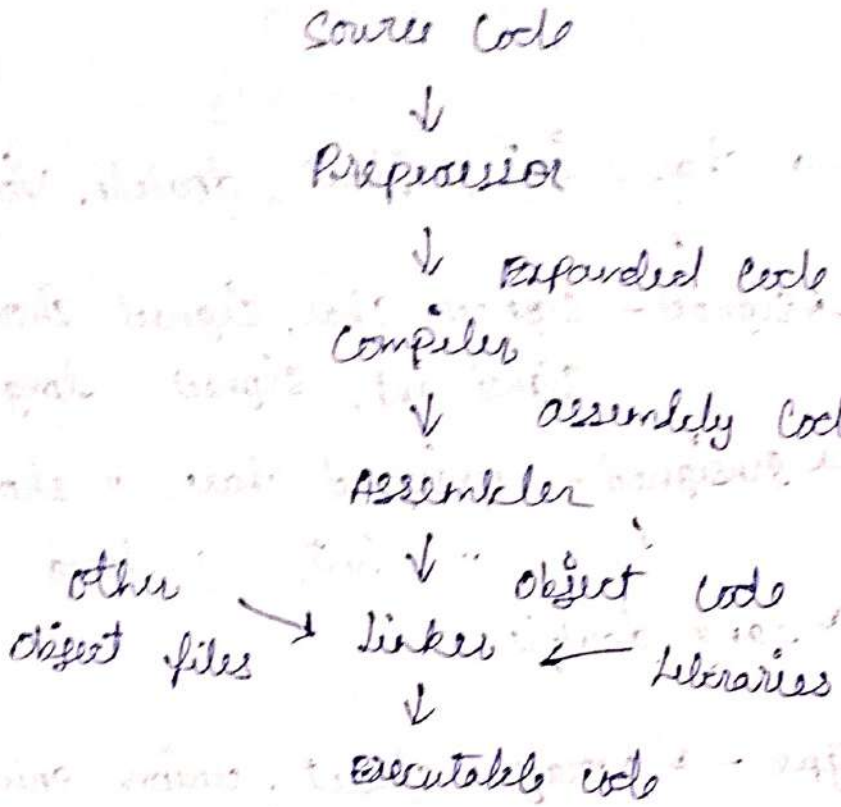
About C

* mother lang
* Sim Programming lang
* Procedure-Oriented Programming lang
* Structured Programming lang
* mid-level lang

Features

* Simple
* Portable
* mid-level
* Structured
* Rich library
* Extensible
* Recursion
* Pointers
* Faster
* memory management

Compile stages in C



C Program Structure

* Documentations
* Pre-Processor Statements
* Global Declarations
* The main() Function
* Local Declaration
* Program Statements and expressions
* User Defined Functions

Header → #include <stdio.h>

main() → int main()
{

Variable declaration → int a = 10;

Body → Printf ("%d", a);

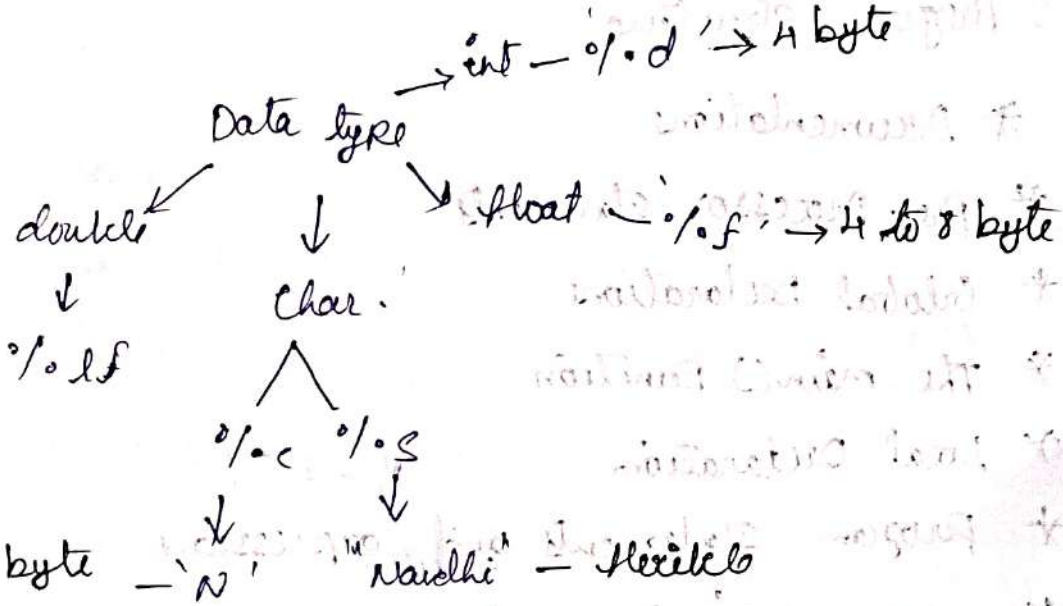
Return → Return 0;
}

Data types :-

primitive type → char, int, float, double, Void

Derived type → Signed - signed char, signed short
signed int, signed long
→ Unsigned - unsigned char, " short,
" int, " long
→ long double

used defined type → Array - struct, union, enum

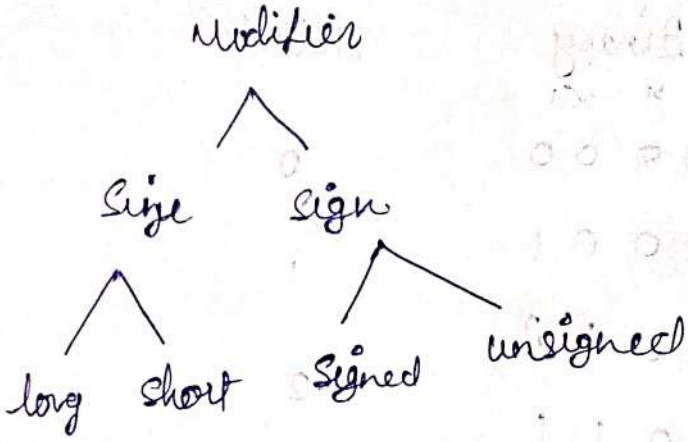


$2^0 - 1$
$2^1 - 2$
$2^2 - 4$
$2^3 - 8$
$2^4 - 16$
$2^5 - 32$
$2^6 - 64$
$2^7 - 128$

$2^8 - 256$
$2^9 - 512$
$2^{10} - 1024$
$2^{16} - 65,536$
$2^{32} - 4,294,967,296$

Desimal	Binary (8 4 2 1)	Octal	Hexa
0	0 0 0 0	0	0
1	0 0 0 1	1	1
2	0 0 1 0	2	2
3	0 0 1 1	3	3
4	0 1 0 0	4	4
5	0 1 0 1	5	5
6	0 1 1 0	6	6
7	0 1 1 1	7	7
8	1 0 0 0	10	8
9	1 0 0 1	11	9
10	1 0 1 0	12	A
11	1 0 1 1	13	B
12	1 1 0 0	14	C
13	1 1 0 1	15	D
14	1 1 1 0	16	E
15	1 1 1 1	17	F

Modifier



In C Programming, the terms modifier & qualifier are often used to describe keywords that change how data type behave. while sometimes used interchangeably in casual discussion, they serve distinct technical purpose.

Data type	Memory size	Range
char	1 byte	-128 to 127
unsigned char	1 byte	0 to 255
Int	2byte	-32,768 to 32,767
unsigned int	2 byte	0 to 65,535
long int	4 byte	-2,147,483,648 to 2,147,483,647
unsigned long int	"	0 to 4294967295
float	"	

Double 8 byte

Long Double 10 byte

Variables:

These are name (identifiers) Associated with locations in memory for storing variable data.

data_type variable_Name = value;

Ex:

```
int age = 25;
int float = 0.25;
int grade = 'A';
```

Rules:-

DO's

* lower case - Ex: a, z or name=1
* upper case - Ex. A, Z or NAME = 1
* Both upper & lower - Ex: Name=a
underscore (_) max_ value = 1
* Variable after number a=10,
a1=3

Don'ts

* Blank (white space not allowed
* Keywords not allowed (int = 10)
* special symbols (!@#$%...)
* can't start with digit Ex: 1=10, 2agid=10.

Double — 8 byte

Long Double — 10 byte

Variables:-

* These are Name (identifiers) Associated with locations in memory for storing variable data.

data_type variable_Name = value;

Ex:

```
int age = 25;
int float = 0.25;
int grade = 'A';
```

Rules:-

Do's

* lower case - Ex: a,..z or name=1
* upper case - Ex. A, Z or NAME=1
* Both upper & lower - Ex: Name=a
underscore (_) max_value=1
* Variable after number a=10,
a1=3

Don'ts

* Blank (white space not allowed
* Keywords not allowed (int=10)
* special symbols (!@#$%...)
* can't start with digit
Ex: 1=10, 2age1=10.

Double 8 byte

Long Double 10 byte

Variables:

These are Name (identifiers) Associated with locations in memory for storing variable data.

data_type variable_Name = value;

Ex:

```
int age = 25;
int float = 0.25;
int grade = 'A';
```

Rules:-

Do's

* lower case - Ex: a,..z or name=1
* upper case - Ex. A, Z or NAME = 1
* Both upper & lower - Ex: Name = a
underscore (_) max_value = 1
* Variable after number a=10,
a1=3

Don'ts

* Blank (white space not allowed
* Keywords not allowed (int = 10)
* Special symbols (!@#$%...)
* Can't start with digit Ex: 1=10, 2age id=10.

Keywords in C

* It is a reserved word. You cannot use it as a variable name, constant name, etc

* there are only 32 Reserved keywords in C lang

* Ex: auto, break, double, float, int, char, long, short, do, if ...

- Declaration of constant in C

* syntax

Const data_type variable_name = Constant;

Ex:

Const float PI = 3.141;

define Preprocessor directives

syntax

define Constant_Identifier Value

Ex:

define PI 3.141

Keywords in C

* It is a reserved word, you cannot use it as a variable name, constant name, etc

* there are only 32 Reserved keywords in C lang

* Ex: auto, break, double, float, int, char, long, short, do, if ....

- Declaration of constant in C

* syntax

Const data_type variable_name = Constant;

Ex:

const float PI = 3.141;

define Preprocessor directives

syntax

define Constant_Identifier

Value

Ex:

define PI

3.141

Storage classes

Auto

* Default storage class for local variable. The value is garbage if not initialized.
* Block level scope
* Created - block entered
  destroyed - block exit
* Default initial value Garbage
* Stack is memory location

Ex:
```
void fun(){
  auto int a = 10;
}
```

extern

* used to declare a global variable define in another file. We can call the function from one file to another by using this keyword.
* File level scope
* Entire Program execution - Lifetime
* Default value - 0
* Memory location - Data

Ex: extern int g; // g is defined in another file.

: static (local)

* A local variable that retains its value b/w the blocks. It is initialized only once
* Block level scope
* Entire program
* Default value – 0
* memory location - Data

Ex: void func(){
    static int count = 0;
    count++;
}

static (global)

* A global variable with internal linkage. It can accessable only within the same file
* File level
* Entire Program
* 0
* Data

Ex: static int g = 5;
    void main(){

    }

Register

* suggests to store the variable in cpu register for faster access. Address & cannot be taken

* Block level
* created - block entered
  destroyed - " exit
* Garbage
* CPU

Ex: register int i = 0;

Operators

1) Assignment operators - =, +=, -=, *=, %=, /=
2) Arithmetic " - +, -, %, /, *, ++, --
3) Relational " - ==, !=, >, <, >=, <=
4) Logical " - &&, ||, !
5) Bitwise " - &, |, ^, ~, <<, >>
6) size of operators - size of (data type / variable)
7) Conditional " - condition ? true exp : false exp
8) Comma " - ,

Arithmetic

+ → add two operators - x+y
- → subtract " " - x-y
* → multiply " " - x*y
/ → Divided " " - x/y
% → modulus remainder of the division - x%y

Ex: #include <stdio.h>

```
int main(){
    int a = 5;
    int b = 6;
    Printf ("%d\n", a+b);
    Printf ("%d\n", a-b);
    Printf ("%d\n", a*b);
    printf ("%d\n", b\a);
    Printf ("%d\n", b%a);
}
```

o/p:
11
-1

Assignment operator :-

= → Assign value → x = value

+= → Add & assign → x = x + value

-= → Subtract & assign → x = x - value

*= → multiply " " → x = x * "

/= → Divide " " → x = x / value

Ex: #include <stdio.h>

```
int main(){
    int a = 9;
    a += 4;
Printf ("%d\n", a);
    a -= 2;
Printf ("%d\n", a);
```

```
a *= 3;                    o/p:
printf("%d\n", a);          13
  a /= 2;                   11
printf("%d\n", a);          33
  return 0;                 16
}
```

Relational operator

`==` → equal to

`!=` → Not equal to

`>` → Greater than

`<` → Smaller than

`>=` → Greater than equal to

`<=` → Smaller than

Ex:
```
int d = 10;
int e = 5;
printf("%d\n", d==e);
printf("%d\n", d!=e);
printf("%d\n", d>=e);
printf("%d\n", d<=e);
printf("%d\n", d<e);
```

o/p

```
0
1
1
0
0
```

Logical Operators

&& → Logical AND → Returns true (1) only if both are true

|| → OR → Return true if least one of the value is true

! → NOT → Reverse the logical state

Ex: #include <stdio.h>
int main {
int d=10;
int e=5;
int f=3;
Printf ("%d\n", d>=e && f>=e);
Printf ("%d\n", d<e && d>=f);
Printf ("%d\n", d>=e || f<=e);
Printf ("%d\n", d<e || d>=f);
Printf ("%d\n", !(e>f));

O/P

Bitwise

Bitwise AND → & - a&b → Returns 1 only if both are true

" OR → | - a|b → " at least one has true

" NOT → ~ - ~b → returns the complement of a bit

" XOR → ^ - a^b → returns 0 if both side are same else 1

" left shift → << - a<<n → shift a towards left by n

Right shift ⟶ >>

& ⟶ Bitwise AND

A	B	A & B
0	0	0
0	1	0
1	0	0
1	1	1

| ⟶ Bitwise OR

A	B	A | B
0	0	0
0	1	1
1	0	1
1	1	1

^ ⟶ XOR

A	B	A ^ B
0	0	0
0	1	1
1	0	1
1	1	0

~ ⟶ NOT

A	~A
0	1
1	0

```
#include <stdio.h>
int main(){
    int a=10;
    int e=5;
    int f=3;
printf("%d\n", e&f);
"  ("%d\n", e|f);
"  ("%d\n", ~f);
"  ("%d\n", e^f);
"  ("%d\n", e<<1); -> multiply
"  ("%d\n", e>>1); -> divided
```

o/p

1
7
-11
6
10
2

e = 5 → 0 1 0 1
f = 3 → 0 0 1 1

0 1 0 1 | 0 1 0 1
0 0 1 1 | 0 0 1 1
0 0 0 1 → 1 | 0 1 1 1 → 7

Left shift ... Right shift

e = 5

0 1 0 1
1 0 1 0 → 10

Unary operator

Ex:

```
Printf("%d\n", f);
Printf("%d\n", f++f);
Printf("%d\n", f++);
Printf("%d\n", f);
```

o/p

3
4
4
2

Unary Plus	→	+
" minus	→	-
Increment operator	→	++
Decrement operator	→	--

Conditional operator (Ternary)

Variable = Exp1 ? Exp2 : Exp3

Variable ← Resultant value
Exp1 ← Expression

Ex: int g = (d>e)? d : e; o/p
Printf ("%d", g); 10

Comma Operator

Syntax:

exp1, exp2, exp3. ... expN

Ex:

```
#include <stdio.h>
int main(){
int a=1, b=2, c;
c = (a=10, b=20, a+b);
Printf("c = %d\n", c);
return 0;
}
```

o/p
30

Sizeof operator

Syntax:

Sizeof (type)
Sizeof (variable/exp)

Ex:

```
#include <stdio.h>
int main(){
int a; float f; double d;
char ch; int arr[10];
Printf ("Sizeof (a) = %zu\n", sizeof (a));
Printf (" " "(f) = " " , " " (f));
```

```
printf ("size of (arr) = %zu \n", sizeof (arr));
    return 0;
}
```

o/p

(a) → 4
(f) → 4
(d) → 8
(ch) → 1
(arr) → 40
(10 * sizeof (int))

user i/p

Ex: int age;
printf ("Enter your age");
scanf("%d", &age);
printf ("your age is %d", age);

o/p

Enter your age : 21
your age is : 21

Control statement

* It controls the flow of a execution
* Sequential statements execute in a linear order, one after another, without any branching
* Instead of going line by line they control what runs, when, and how many times

Flow of control

* Sequential stmts
* Selectional / Conditional stmt
* Loop / iterative / Repetitive stmt
* Jumping / transfer stmt

Conditional

* choose one path based on a condition
* condition → true / False

i) If stmt

syntax:

```
if (condition){
  // block of code to be executed if the condition true}
```

Ex:

```
#include <stdio.h>
int main (){
int a = 10;
if (a<20){
  Printf (" a is greater less than 20");
}
}
```

O/P

a is less than 20

Else block

```
int a = 10;
int b = 40;
```

~~Printf (~~

```
if (a>b){
   Printf ("a is greater than b");
  } else{
   Printf ("b is greater than a");
}
```

syntax:

```
if (condition){
   // executed only the condition is true//
  } else {
   // executed if the condition is false //
}
```

else if stmt

else if stmt to specify a new condition if the first condition is false.

syntax

```
if (Condition 1){
   // executed if condition is true }
else if (Condition 2){
   // if the condition 1 is false & con2 is true }
else {
  // executed if con is false//
}
```

Ex: int b = 70;

o/p

charge later

```
if (b<20){
  Print ("charge now");
} else if (b<60){
  Print ("power saving mode on");
} else {
  Print ("charge later");
}
```

Nested if → The nested if that represents the if block within another if block, then the inner if block condition.

Syntax:

```
if (condition){
  // code to be executed
  if (condition){
    // code to be executed
  }
}
```

Ex:

o/p

charge now charge immediately

```
int b = 10;
if (b<20) {
  Print ("charge now");
}
if (b==10) {
  Print ("charge immediately");
}
```

Ex: int b = 70;

o/p

charge later

```
if (b<20){
  Printf ("charge now");
} else if (b<60){
  Printf ("Power saving mode on");
} else {
  Printf ("charge later");
}
```

Nested if → the nested if statement represents the if block within another if block, here, the inner if block condition is written.

Syntax:

```
if (condition){
  // code to be executed
  if (condition){
    // code to be executed
  }}
```

Ex:

o/p

charge now charge immediately

```
int b = 10;
if (b<20) {
  Printf ("charge now");}
if (b==10) {
  Printf ("charge immediately");}
}
```

Switch stmt

It executes one stmt from multiple condi. It is like if-else-if ladder stmt

Syntax

```
switch (expression)
{
case value 1:
//code to be executed;
break; //optional
default:
code to executed if all cases are not matched;
}
```

Ex:

```
int day = 1;
switch (day)
{
case 1:
    Printf ("monday \n");
break;
case 2:
    Printf ("thursday \n");
break;
    :
case 5:
    Printf ("Friday \n");
break;
default:
    Printf ("weekend");
}
```

o/p

Monday

Switch stmt

It executes one stmt from multiple conditions like if-else-if ladder stmt

Syntax

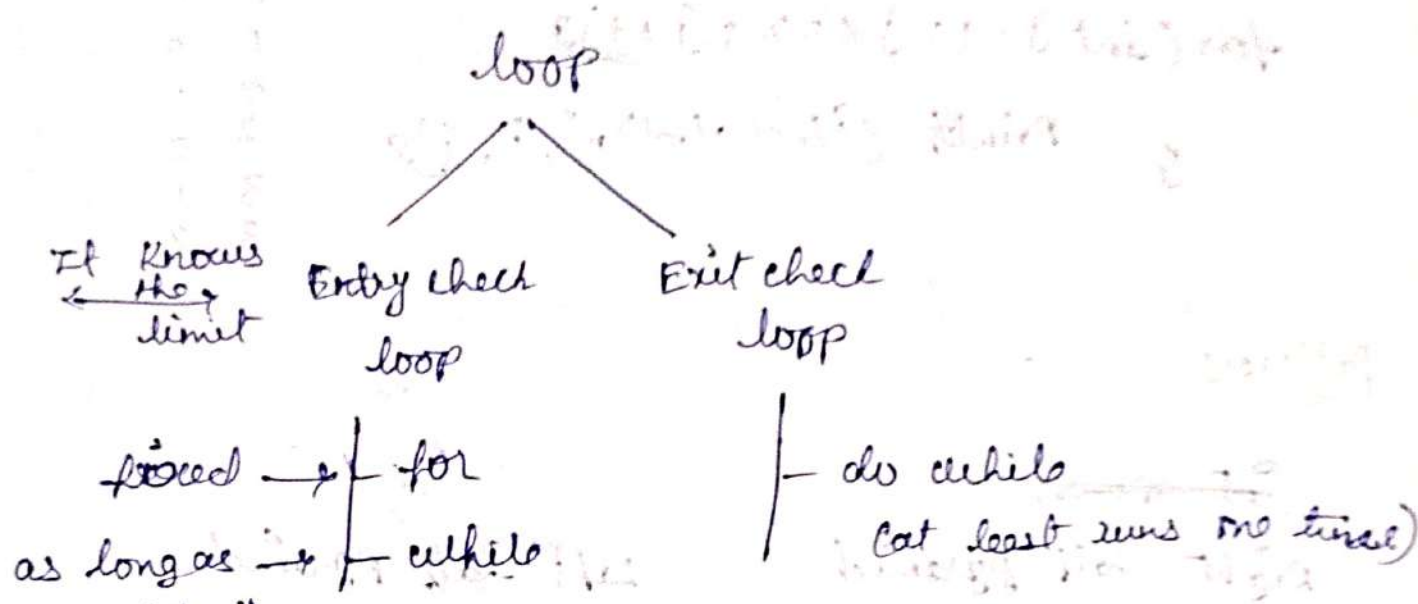
```
switch (expression){
case value 1:
//code to be executed;
break; //optional
default:
code to executed if all cases are not matched;
}
```

Ex:
```
int day = 1;
switch (day){
case 1:
    Printf ("monday \n");
break;
case 2:
    Printf ("tuesday \n");
break;
    :
case 3:
    Printf ("Friday \n");
break;
default:
    Printf ("weekend");
}
```

o/p
Monday

Loop / iterative / repetitive statement

It is used to execute a particular part of the program repeatly, if a given condi evaluates to be true



for loop

Syntax

```
for (initialization; condition; incre/decre)
{
  //stat or code
}
```

Ex:

```
for (int i = 0; i<=5; i++){ → o/p: 0 1 2 3 4 5
    Printf ("%.d", i);
}
for (int i = 10; i>=5; i--){ → o/p: 10 9 8 7 6 5
    Printf ("%.d", i); }
```

```
for(;;){
    Printf("all is well");
}
```

o/p → all is well (infinity times)

nested for

```
for(int i=1; i<=3; i++){
    for(int j=1; j<=2; j++){
        Printf("%d %d\n", i, j);
    }
}
```

o/p →

```
1 1
1 2
2 1
2 2
3 1
3 2
```

Patterns

~~int n=5;~~

Right half Pyramid

```
*
* *
* * *
* * * *
* * * * *
```

Left half Pyramid

```
    *
  * *
* * *
```

full Pyramid

```
    *
  * * *
* * * * *
```

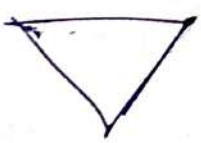
inverted Right half Pyramid

```
* * * *
* * *
* *
*
```

inverted left half Pyramid

```
* * * *
 * * *
  * *
   *
```

Inverted full Pyramid



Types of while loop

while loop - increment

while loop - decrement

Infinity while loop

Ex: while loop

```
int i = 0;
while (i<=10){
  Printf ("the numbers are %-d\n", i);
  i++;
}
```

```
int i = 10;
while (i>=5){
  Printf ("the numbers are %-d\n", i);
  i--;
}
```

```
while (1){
  Printf ("hi Karthika");
}
```

→ O/P : Printf infinity times

Do-while loop

If the number of iteration is not fixed & you must have to execute the loop at least once, it is recommended to use do-while loop

Syntax

```
do
{
 // code executed (loop body
 // update stmt
}while (condi);
```

Ex:
```
int i=5;
do{
  Printf("hello %d\n", i);
  i++;
}while (i<20);
```

1) Break

It stops the stmt

Ex:
```
for(int i=5; i<=10; i++){
  if(i==8){
    break;
  }
  printf("%d, i);
}
```

o/p
5 6 7

2) Continue

It skip the stmt and continue

Ex.
```
for(int i=5; i<=10; i++){
  if(i==8){
    continue;
```

o/p
5 6 7 9 10

```
    }
    printf("%d", i);
}
```

3) goto start

It is known as jump start in C. as the name suggests, goto is used to transfer the program control to a predefined label. the goto start can used to repeat some part of the code for a particular condi

Ex:

```
int main(){
int i = 0;
start: // label for goto
for(i=0; i<5; i++){
if(i==1){
 continue; // skip 1
}
if(i==3){
 break; // stop loop at 3
}
static int flag = 0;
if(flag==0){
flag=1;
printf("\n Repeating using goto:\n");
goto start;
} return 0; }
```

String :

* the string can be defined as the one-dimensional array of characters terminated by a null ('\0')

* Each character in the array occupies one byte of memory. & the last character must always be 0

* the termination character ('\0') is important in a string since it is the only way to identify where the string ends.

* there are two ways to declare a string :

1) By char array
2) By string literal

1) Declaring string by char array

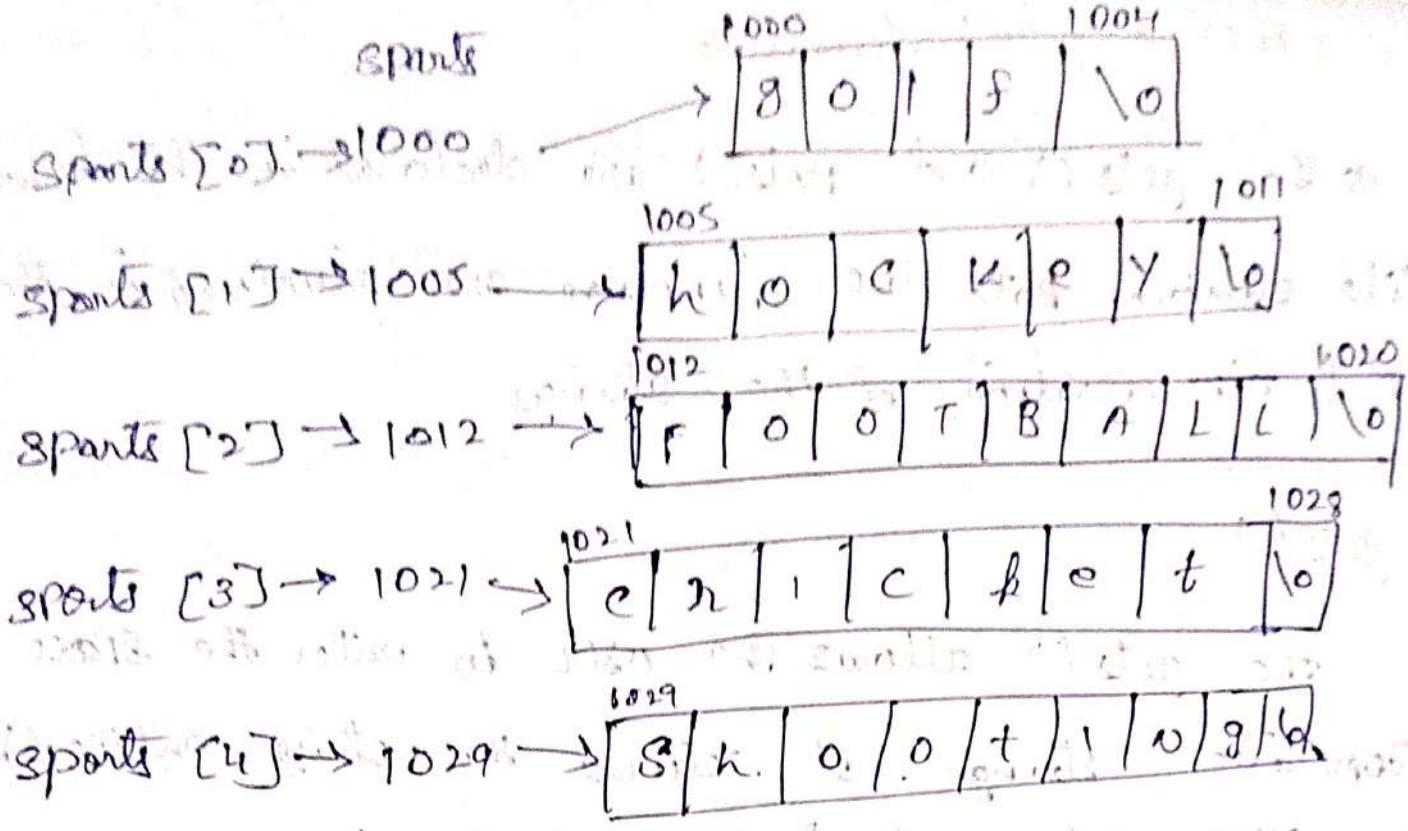
```
char ch [10] = {'N', 'A', 'N', 'D', 'H', 'I', '\0'};
```

0	1	2	3	4	5	6
N	A	N	D	H	I	\0

2) string by the string literal

```
char ch [] = "Hello C";
```

in such case, '\0' will be appended at the end of the string by the compiler.



Accepting string as the i/p

* Till now, we have used scanf to accept the i/p from the user
* consider the below code which stores the string while encountered

```
#include <stdio.h>
void main()
{
    char name[10];
    Printf("Enter your name");
    scanf("%s", name);
    Printf("your name is ms. %s", name);
    return 0;
}
```

C gets() & puts() functions

* the gets() and puts() are declared in the header file stdio.h. Both are functions are involved in the i/p & o/p operation of the strings

gets()

the gets() allows the user to enter the space separated strings. it returns the string entered by the user. it used instead of scanf

Syntax:

```
gets ( string_name, size, stdin);
```

Ex:

```
int main() {
    char name[30];
    printf("enter your name");
    gets (name, sizeof(name), stdin);
    printf("your name is ms. %s", name);
    puts(name); → instead of printf
}
```

o/p:

enter your name naren arya

your name is naren arya

String function

1) strlen (string_name) → returns the length of string name

2) strcpy (destination, source) → copies the content of source string to destination string

3) strcat (first_string, second_string) → concats or joins 1st string with 2nd string. the result of the string is stored in 1st string

4) strcmp ("1st_string, 2nd_") → compares the 1st string with 2nd string. if both strings are same, it return 0

#include <string.h> => It should be used when we use string header
#include <stdio.h>

Ex:-
```
int main() {
char str[20]; char str2[15] = "word";
char str1[15] = "word";
char str2[20] = "encyclopedia";
Printf ("%c\n", str2[2]);
Printf ("%d\n", strlen(str2));
printf ("%s\n", strcat (str1, str2));
Printf (" before copy\n", str);
strcpy(str, str2);
Printf (" after copy %s\n", str);
// strings compare
if (strcmp(str1, str2)==0) {
    printf ("strings are equal");
} else {
```

o/p
c
12
wordencyclopedia
before copy
after copy encyclopedia

Printf ("strings are not equal");

o/p
strings are eq

{

upper & lower case function

#include <string.h>
#include <stdio.h>
#include <ctype.h> → this header used for upper, lo case functio

int main(){
char str[] = "universe."; → here we can mention the size or not. otherwise it will take the size when it empty

for (int i=0; str[i]!='\0'; i++){
str[i] = to upper (str[i]);
} str[i] = to lower (str[i]); → this is for lower case

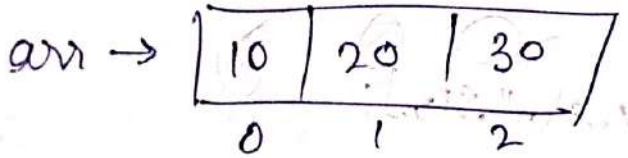
Printf ("%s", str);

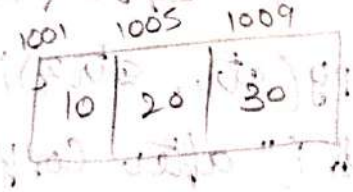
o/p
UNIVERSE

o/p
universe

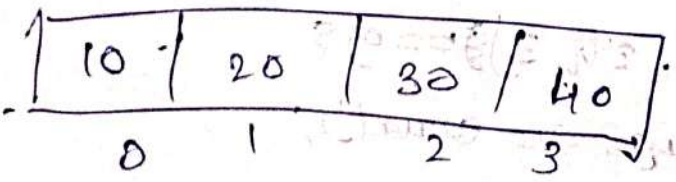
Arrays in C

An array is a collection of elements of the sam Data type stored in contiguous memory locations





Accessing elements

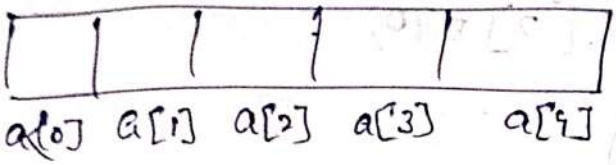


arr[i] = 20

Declaration

int a[5];

Creat space for 5 integers



Types of arrays

1) 1D array

int nums[4] = {50, 15, 20};

5	10	15	20

2) 2D Array (matrix)

int matrix [2] [2] = {{1,2}; {3,4}};

1	2
3	4

10	20	30	40

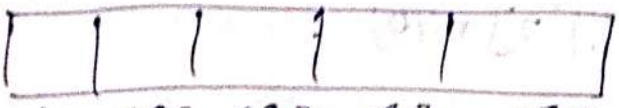
(i) → 1001 + (0 × 4) = 1001

address of i = Base address + (index × data type bytes)

Declaration

int a[5];

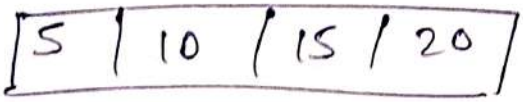
Creat space for 5 integers



Types of arrays

1) 1D array

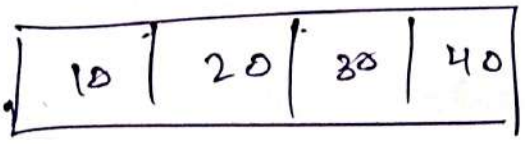
int nums[4] = {50, 15, 20};



2) 2D Array (matrix)

int matrix [2] [2] = {{1,2}; {3,4}};

1	2
3	4



① → 1001 + (0 × 4) = 1001

address of i = Base address + (index × data type bytes)

Single dimensional array

```
void main(){
int a[5];
a[0]=10; a[1]=20; a[2]=70;
for( int i=0; i<4; i++)
Printf("%d", a[i]);
}}
```

multidimensional array

```
void main(){
int arr[3][3]={{1,2,3},{2,4,5},{4,4,5}};
// Printing 2D array.
for(int i=0; i<3; i++){
  for(int j=0; j<3; j++){
    Printf("%d", arr[i][j]);}
    Printf("\n");
}}
```

Arrays are used in:

1) DMA Buffers
2) ADC sample storage
3) UART ring buffers
4) sensor batching

```c
#include <stdio.h>
int main(){
  int arr[5], i;
  Printf("enter your values");
  for(i=0; i<5; i++){
    Scanf("%d", &arr)->arr[i]
  }
  Printf("display array values");
  for(i=0; i<5; i++){
    Printf("%d", arr)->arr[i]
  }
  return 0;
}
```

O/P

enter your values

1

2

3

4

5

display array values

1592159... some address values will Print

→ O/P

12345 it is normal array print

Pointers

* the pointer in c language is a variable which stores the address of another variable

* consider the following example to define a pointer which stores the address of an integer

1) int n = 10;

2) int * p = &n; // variable p of type pointer is Pointing to the address of the variable n of type integers

* the pointer in c lang can be declared using * (asterisk symbol). it is called as indirection pointer used to dereference a pointer

i) int *a; // pointer to int

ii) char *c; // pointer to char

Ex:

```
int a=10;
int *p;
p= &a;
Printf("Value of a = %d\n", a);
"("address of a = %p\n", &a);
"("Pointer value = %p\n", p);
"("Value using pointer = %d\n", *p);
```

o/p

Value of a = 10
Address of a = x0 ffe0d637
Pointer value = " "
Value using pointer = 10

//array-pointer

```
int arr[3]={10,20,30};
Printf("%d\n", arr[0]);
"("%d\n", *(arr+0));
"("%d\n", *(arr+1));
```

o/p

10
10
20

we can call the a value by using pointer

Functions in C

* the function or method is a collection of instructions that perform a specific task

* you can pass data, known as parameters, into a method

* function is a block of code which runs only it is called

Ex: switch have 2 functions that is on() & off()

why function

* It provides the reusability of code

* It also provides the easy modification & readability of code, just by adding or removing a chunk of code

* function are useful in hiding the implementation details

* It is easier to edit a program, which is written using method

Types :-

* Predifined method * Recursive "

* user-defined "

* Return "

Predefined method

1) Built → #include <stdio.h> → to get i/p & o/p
2) String → #include <string.h> → strlen(), strcat()
3) toupper(), tolower(), isalpha(), isdigit(), isalnum() → #include <ctype.h>

* It is also called as the standard library meth
or built-in method

* some pre-defined methods are length(), equ
compareTo(), sqrt(), etc.

user-defined method

* the method written by the user or programmer
called as a user-defined method

* these methods are modified according to the
requirement

* Types

1) Function with argument, with return valu
2) " " " , without " "
3) " without " , with " "
4) " " " , without " "

syntax for function :

Return- type function name (datatype variable /argum

```
{
 function body;
}
```

Ex: without parameters, without return

```
#include <stdio.h>
void add(){
int a=2, b=3;
Printf ("sum =%d\n", a+b);
}
 int main(){
add();
return 0;
}
```

// with p, without r

```
void add ( int a, int b){
Printf ("sum = %d\n", a+b);
{
 int main() {
 add (20, 30);
 return 0;
}
```

```
// without P, with x
int add(){
int a = 3, b = 3;
return a+b;
}
int main(){
int a = add();
Printf("%d", a);
return 0;
}
```

→ function block

```
// with P, with x
int add(int a, int b)
return a+b;
}
int main(){
int a = add(10,20);
Printf("%d", a);
return 0;
}
```

Recursion in C (it starts with the block ends):

* These in java is a process in which a method calls itself continuously
* A method in java that calls itself is called recursive method

Syntax:

```
Return type method name(){
method name(); // calling same method
}
```

Ex:

```
#include <stdio.h>
int fac(int n){
if(n==1){
return 1;
else
```

```
>return (n* fac (n-1));
}
void main()
{
Printf ("factorial of 5 is: %d", fac (5));
}
```

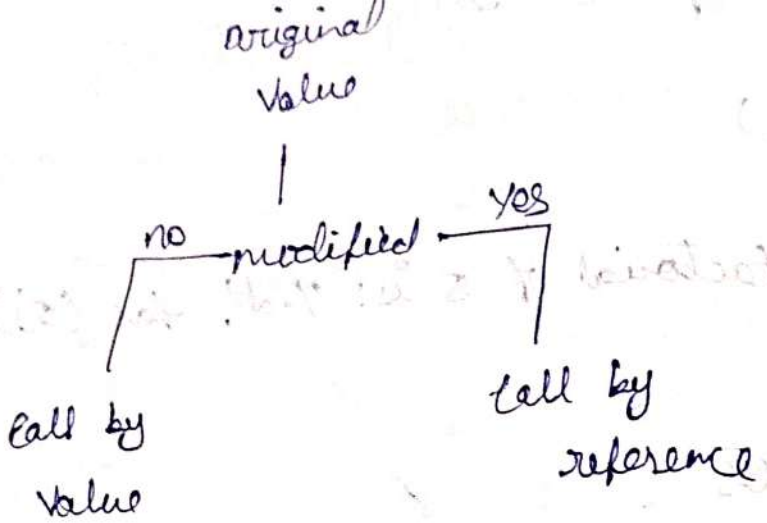
multiply Tables

```
#include <Stdio.h>
int main(){
  int a;
printf ("enter your num: ");
scanf ("%d ", &a);
for (int i=1; i<=10; i++){
    printf ("%d x %d = %d\n", i, a, i*a);
}
  return 0;
}
```

Function parameter types of passing

1) call by value and call by reference in c

there are two methods to pass the data into the function in c lang, i.e., call by value & call by reference



Call by value C

* The value of the actual parameter is copied into the formal parameters. In other words, we can say that the value of the variable is used in the function call in the call by value method.

* In call by value method, we can not modify the value of the actual parameter by the formal parameter.

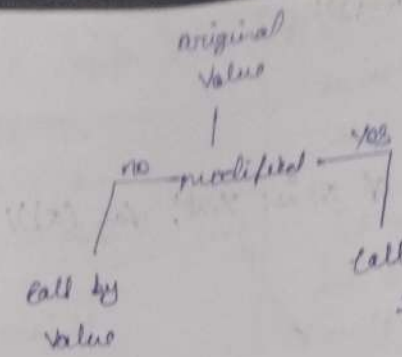
Ex:

```
# include <stdio.h>
void change(int x){
    x=100;
}
int main(){
    int a=10;
    change(a);
```

change into a

= it takes only the

a value can't change



call by value :-

* The value of the actual parameter is copied into the formal parameters in other words, we can say that the value of the variable is used in the function call in the call by value method.

* In call by value method, we can not modify the value of the actual parameter by the formal parameter.

Ex:

```
# include <stdio.h>
void change (int x){
    x=100;
}
int main(){
    int a=10;
    change (a);
```

printf ("%d", a);
return 0;
}

call by reference

* the address
function call as

* the value of
modified by chan
the address of th

* In call by
is similar for
para.

Ex:

```
void change
    *x = 100 ;
}
int main()
    int a = 10
change (&a
Printf ("%
return 0;
}
```

```
printf("%d", a); // child 10
return 0;
}
```

o/p
10

call by reference in C

* the address of the variable is passed into the function call as the actual parameter.
* the value of the actual parameters can be modified by changing the formal parameters since the address of the actual parameter is passed
* In call by reference, the memory allocation is similar for both formal para & actual para.

Ex:

```
void change (int *x) {
    *x = 100;
}
int main(){
    int a = 10;
    change (&a);
    printf("%d", a)
    return 0;
}
```

→ Here we use Pointer (*) so the address operator takes the a address & pointer change the value of a into x
So the o/p is 100

→ Here we use address operator. So it take the a address

Ex:

```
#include <stdio.h>
void main(){
  int a=10, b=20;
  Printf("Before swaping the numbers are %od, %od\n",
                                    swap(&a,&b);
  Printf("after swapping values are.. %od, %d", a, b);
}
  void swap(int *x, int *y)
  {
*x = *x + *y;
*y = *x - *y;
*x = *x - *y;
}
```

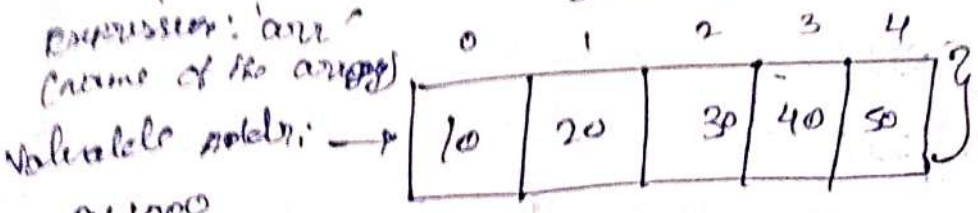
Pointer

Declaration of pointer

1) int *p; → Pointer to int
2) char *c; → Pointer to char
3) float *f; → " to float
4) int **pp → " to pointer

Pointer in depth : arr Vs. &arr

int arr[5]



Type: int *
(Pointer to an integer)

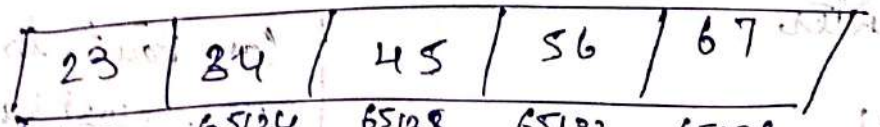
Type: int (*)[5]
(Pointer to an array of 5 integer)

arr + 1 = address of 2nd element
0x1004
0x1004
moved by size of (int) (4 byte)

&arr + 1 = Address after the entire array
0x1014
moved by (sizeof (arr)
(20 bytes)

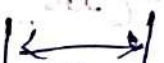
int ary[5] = {23, 34, 45, 56, 67}; Array Pointer:

int *p = ary;



*p *(p+1) *(p+2) *(p+3) *(p+4)

→ memory location



Pointer arithmetic

Ex: #include <stdio.h>

int main(){

int arr[5] = {10, 20, 30, 40, 50};

int *ptr = &arr[2];

ptr = ptr + 2; printf("%d", *ptr);

Sub → (ptr = ptr - 2); return 0; }

```
int arr[] = {10,20,30,40};
int *p1 = &arr[0];
int *p2 = &arr[3];
Printf("%ld", p2-p1);
return 0;
}
```

→ here pointer to pointer can subtract but not to add but can sub & add by using pointers

Types of pointer.

1) Null Pointer

```
int *p = Null;
if (p == NULL){
  Printf("p is a Null Pointer");
}
```

Points to no valid memory address. used to initialize Pointers

2) Void Pointer

```
Void * ptr;
int x=10;
Ptr = &x; int y = *(int *) ptr;
```

Can Point to any data type. must be cast before dereferencing.

Ptr → int / float / char

3) Wild pointer

```
int *WP;
*WP = 20; // Dangerous
```

Declared but not initialized. Points to an arbitary location. very dangerous (here we don't assign the value for the pointer which is not declared)

4) Dangling pointer

```
int *p;
free(p);
*p = 20;
```

free(p) is used to create a free space in bytes so here we use free(p) method so it let the bytes free, so that we can't assign the value in free(p) it's useless.

5) constant pointer

```
int x = 10;
int * const p = &x;
*p = 20; // allowed
```

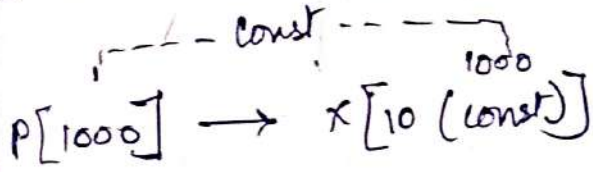
Pointer itself is unchangeable, but the value it points to can be modified.

6) Pointer to constant

```
int y = 10;
const int *p = &y;
p = &z; // allowed
```

Value the pointer points to is unchangeable (const), but the pointer can pointer to different data.

7) constant pointer to constant




```
Const int * const p = &num;
*p = 15; // error
p = &num2; // error
```

Both pointer and data are constant. Pointer itself is unchangeable, and the data if points to cannot be modified

8) function pointer

```
void (*func_ptr)(int, int);
func_ptr = &add;
(*func_ptr)(5,7); // calls add
```

Pointing to the starting address of executable code in memory

9) Double pointer (Pointer to pointer)

```
int x = 10;
int *p = &x;
int **pp = &p;
Printf("%d", **pp); // Print 10
```

Stores the memory address of another pointer variable

Pointer utilities

a) Null Pointer

A Null pointer does not point to any valid memory location

```
int *p = NULL;
if (p == NULL)
{
  Printf("No address assigned");
}
```

o/p: No address assigned

b) size of()

Returns the size of a pointer (size of an address)

```
int x;
int *p = &x;
Printf("%zu", sizeof(p));
```

o/p
32-bit system: 4
64-bit sys: 8

Mastering memory manipulation in C

1) memcpy() → copies memory blocks efficiently in C
2) memcmp() → compares memory blocks for equality
3) memmove() → safely moves overlapping memory blocks
4) memset() → sets memory blocks to a specific value.

1) memcpy():

Copies a specified number of bytes from a source memory location to a destination memory location. It is fast but should not be used when the source & destination overlap.

Syntax:

```
void * memcpy ( void *dest, const void * src,
                size_t n);
```

Ex:

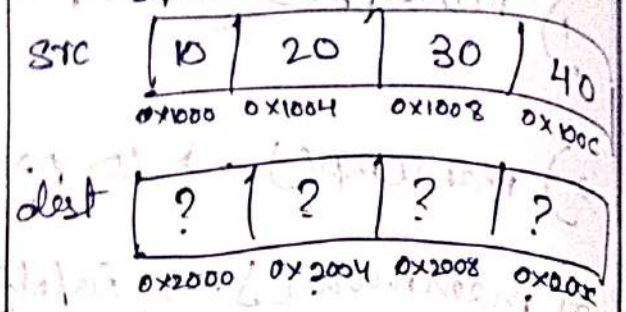
```
#include <stdio.h>
#include <string.h>
int main(){
    int src[] = {10, 20, 30, 40};
    int dest[4];
    memcpy(dest, src, sizeof(src));
```

```
Printf ("Copied Array: ");
for(int i=0; i<4; i++)
    Printf ("%d", dest[i]);
    return 0;
}
```

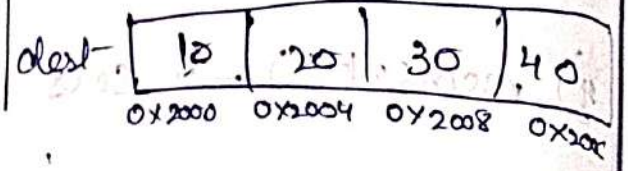
O/P

Copied Array: 10 20 30 40

Before memcpy()



After memcpy()



2) memmove()

Copies a specified number of bytes from one memory location to another. Unlike memcpy(), it works correctly even if the source & dest memory overlap

Syntax:

```
void *memmove (void *dest, const void * src,
                size_t n);
```

Ex:

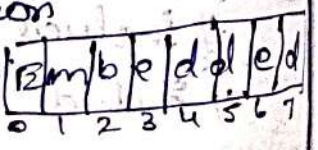
```
char str[] = "Embedded";
memmove (str+2, str, 5);
Printf ("%s", str);
return 0;
}
```

O/P

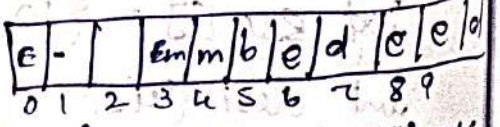
EmEmbedded

Explanation

original:



after memmove (str+2, str, 5)



The 1st 5 charac ("Embed") are copied to str+2 safely, even though the memory regions overlap.

3) memcmp()

compares two memory blocks byte by byte for a specified number of bytes.

syntax:

```
int memcmp (const void *ptr1, const void *ptr2,
            size_t n);
```

Ex:

```
char a[] = "ABC";
char b[] = "ABc";
if (memcmp (a, b, 3) == 0)
    Printf ("Equal");
else
    Printf ("Not equal");
return 0;
}
```

o/p

Equal

Return value

Return value	meaning
0	memory blocks are equal
<0	1st block is smaller
>0	1st block is greater

4) memset()

Fills a block of memory with a specified byte value

syntax:

```
void * memset (void
        * ptr, int value,
            size_t n);
```

Ex:

```
char str[10];
memset (str, 'A', 5);
str[5] = '\0';
Printf ("%s", str);
return 0;
```

AAAAA

before memset (str, 'A', 5)

index	0	1	2	3	4	5	6	7	8	9
str:	?	?	?	?	?	?	?	?	?	?

↓

after memset (str, 'A', 5) & str[5] = '\0'

index:	0	1	2	3	4	5	6	7	8	9
str:	A	A	A	A	A	\0	?	?	?	?

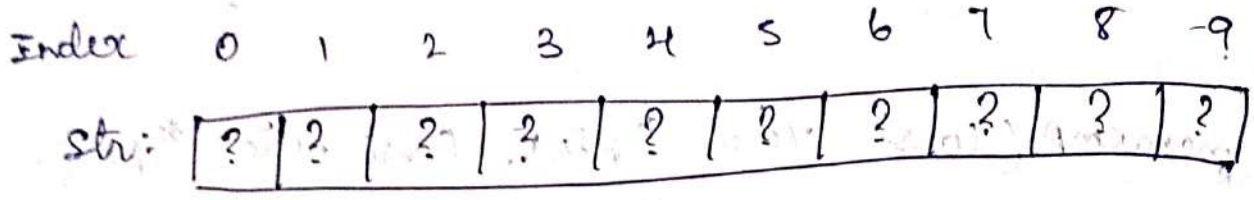
Static & Dynamic memory allocation

static	Dynamic
1) Allocated at compile time	1) Allocated at runtime
2) uses stack / data segment	2) uses heap
3) Fixed size	3) Flexible size
4) Automatically released when out of scope	4) must be manually released using free()

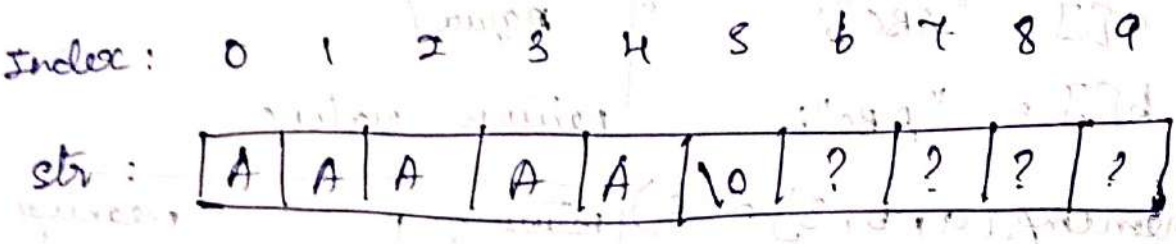
O/P

AAAAA

Before memset (str, 'A', 5)



After memset (str, 'A', 5) & str[5] = '\0'



Static & Dynamic memory allocation

static	Dynamic
1) Allocated at compile time	1) Allocated at runtime
2) uses stack / data segment	2) uses heap
3) Fixed size	3) Flexible size
4) Automatically released when out of scope	4) must be manually released using free()

Stack

stores → Local variables,
method calls

access → only current thread

speed → fast

size → smaller

life → Ends when method ends

management → Automatic (LIFO)

Dynamic

stores → Objects, instance variables

access → shared among all threads

speed → slower

size → larger

life → Exists until GC removes

management - Garbage collector

types :-

1) malloc() → allocates a block of memory

2) calloc() → allocates multiple blocks & initialises to zero

3) Realloc() → Resizes previously allocated memory

4) free() → Frees the allocated memory.

1) malloc()

* allocates a block of memory of the requested size
* memory contents are uninitialised (containing garbage value)
* Returns a void * pointer
* Syntax

ptr = (datatype *) malloc(no. of elements * sizeof(datatype));

Ex:

```
#include <stdio.h>
#include <stdlib.h>   → standard library here only we can use the malloc()...
int main(){
int * ptr = (int *) malloc (5 * sizeof (int));
if (ptr == NULL)
{
printf ("memory allocation failed");   o/p
return 1;                              10 20
}
ptr[0] = 10; ptr[1] = 20;
printf ("%d %d\n", ptr[0], ptr[1]);
free (ptr); return 0; }
```

2) Calloc

* Allocates memory for multiple elements
* Initialises all allocated bytes to zero

Syntax

ptr = (datatype *) calloc (no_of_elements, size of (datatype));

Ex:

```
int main() {
    int * ptr = (int *) calloc (5, sizeof (int));
    Printf ("%d \n", Ptr[0]); // Print 0
    free (ptr);
    return 0;
}
```

3) Realloc

* changes the size of an already allocated memory block
* can increase or decrease the allocated size
* Existing data is preserved (up to the smaller of old & new sizes)

Syntax:

ptr = (datatype *) realloc (Ptr, new_size);

Ex:

```
int main(){
  int *ptr = (int *)malloc(3 * sizeof(int));
  Ptr = (int *)realloc(ptr, 6 * sizeof(int));
    free(ptr);
    return 0;
  }
```

4) free()

* Releases dynamically allocated memory back to the heap

* Prevents memory leaks

Syntax

```
free(Ptr);
Ptr = NULL;
```

Ex:

```
int main(){
  int * ptr = (int *)malloc(sizeof(int));
    free(ptr);
  Ptr = NULL;
  return 0;
}
```

typedef

* In C, typedef is a keyword used to create new name (alias) for an existing data type. It does not create a new type. It simply provides another name for an existing one.

Syntax:

typedef existing_data_type new_name;

Ex:

```
#include <stdio.h>

typedef unsigned int Integer;

int main(){

    Integer x = 10;
    Printf("%d\n", x);
    return 0;
}
```

Advantages:

* make code shorter & easier to read
* simplifies complex declarations (especially pointers & structures)
* Improves portability by allowing type names to be changed in one place

(int *) malloc (3 * sizeof (int));
) realloc (ptr, 6 * sizeof (int));
r);

lly allocated memory back

leaks

ix (sizeof (int));

typedef

* In C, typedef is a keyword used to create a new name (alias) for an existing data type. It does not create a new type. It simply provides another name for an existing one.

Syntax

```
typedef existing_data_type new_name;
```

Ex:

```
#include <stdio.h>

typedef unsigned int Integer;
int main(){
    Integer x = 10;
    Printf("%d\n", x);
    return 0;
}
```

Advantages

* Make code shorter & easier to read
* Simplifies complex declarations (especially pointers & structures)
* Improves portability by allowing type names to be easily changed in one place

what is structure

* It in C is a user-defined datatype that enables us to store the collection of different data
* Each element of a structure is called a mem
structure ca; simulate the use of classes and templ
as it can store various information
* the struct keyword is used to define the struc

Syntax

```
struct structure_name
{
    data_type member1;
        ⋮
    data_type membern;
};
```

1st way :-

* Let see the ex to declare the structure variable by struct keyword. It should be declared either the main function

1)
```
struct employee
{
    int id;
    char name[50];
    float salary;
};
```

2nd way

* Let's see another way to declare variable at time of defining the structure

```
struct employee
{ int id;
  char name[50];
  float salary;
} e1, e2;
```

) struct

Ex:

```
#include <stdio.h>
#include <string.h>
struct student {
  int id;
  char name[50];
  float marks;
};
int main() {
  struct student s1 = {1, "karthi", 10};
  struct student s2;
  s2.id=2; strcpy(s2.name," vaidhiethi"); s2.marks=5;
  Printf("%d %s %f \n", s1.id, s1.name, s1.marks);
  "(" ", s2.id, s2.name, s2.marks);
```

O/P

1 karthi 10.000
2 vaidhiethi 5.0000

without typedef

```
struct student {
    int id;
    char name[20];
};
struct student s1;
```

with typedef

```
# include <stdio.h>
typedef struct {
    int id;
    char name[20];
} student;
int main() {
    student s1 = {101, "Alu
Printf ("ID: %d. Name: %
    , s1.id, s1.
    name
    return 0;
}
```

Union :-

* union can be defined as a user defined data type which is a collection of different variables of different data types in the same memory location

* The union can also be defined as many members but only one member can contain a value at a particular point in time

Syntax:-

```
union tagname{
    data_type member1;
        ⋮
};
union tagname variable 1, variable 2....
```

Ex:-

```
#include<stdio.h>
union student{
    int id;
    int h;
    float w;
};
int main(){
    union student s1;
    s1.id = 10;
    printf("%d\n", s1.id);
    s1.h = 5;
    printf("%d\n", s1.h);
    s1.w = 40;
    printf("%d\n", s1.w);
}
printf("%d\n", s1.id);
```

O/P

```
10
5
40
```

→ If we call the same value again it will print

```
10
5
40
40
```

struct	union
Keyword → the keyword is used to define a structure	* the keyword union is to define a union
* Size → when a variable is associated with a structure the compiler allocates the memory for each member. the size of struct is greater than or equal to sum of size of its members	* when a variable is associated with union the complier allocates the memory by considering the size of the largest memory, so, size of union is equal to the size of largest member
* Memory → Each ~~memory~~ member within a structure is assigned unique storage area of location	* it is shared by individual member of union
* Value Altering → Altering the value of an member will not affect other member of struct	* the value of any of the member will alter other members values
* Accessing member → Individual member can be accessed at a time	* only one member can be accessed at a time
* Initialization of member → several member of a structure can initialize at once	* only the 1st member of a union can be initialized

Struct	union
keyword → the keyword is used to define a structure	* the keyword union is used to define a union
* Size → when a variable is associated with a structure the compiler allocates the memory for each member. the size of struct is greater than or equal to sum of size of its members	* when a variable is associated with union, the compiler allocates the memory by considering the size of the largest member, so, size of union is equal to the size of largest member
* Memory → each member within a structure is assigned unique storage area of location	* it is shared by individual member of union
* Value altering → altering the value of an member will not affect other member of struct	* the value of any of the member will alter other member values
* Accessing member → individual member can be accessed at a time	* only one member can be accessed at a time
* Initialization of member → several member of a structure can initialize at once	* only the 1st member of a union can be initialized

Enumeration

* It used for the fixed choice.

syntax :-

```
enum TagName {
    Constant 1,
    Constant 2,
    constant 3,
    ..  4,
};
```

→ Here we can give any names

```
enum TagName Variable;
```

Ex:

```
enum led {
    ON, OFF, dim
     0,  1,   2
};

int main(){
    enum led s1 = dim;
    Printf("%d", s1);
}
```

→ It will calls the index even if its strings

Preprocessor directives

* The C preprocessor is used by compiler to transform your code before compilation

* It's called micro preprocessor because it allows us to add macros.

1) # include (File Inclusion)

used to include the contents of another file into the current source file.

Syntax:

```
# include <stdio.h> // system header file
# include "myfile.h" // user-defined header file
```

Ex:

```
# include <stdio.h>
int main()
{
    Printf ("Hello World");
    return 0;
}
```

O/P
Hello World

2) # define (macro Definition)

Defines constants or macros.

Syntax

```
# define identifier value
```

Preprocessor directives

* The c preprocessor is used by compiler to transform your code before compilation

* It's called micro preprocessor because it allows us to add macros.

1) # include (File Inclusion)

used to include the contents of another file into the current source file.

Syntax:

```
# include <stdio.h> // system header file
# include "myfile.h" // user-defined header file
```

Ex:

```
# include <stdio.h>
int main()
{
    Printf ("Hello World");
    return 0;
}
```

O/P

Hello World

2) # define (macro Definition)

Defines constants or macros.

Syntax

```
# define identifier value
```

Ex: 1 → constant

O/P

Area = 78.54

```
# include <stdio.h>
# define PI 3.14159

int main(){
    float r=5;
    printf("Area = %.2f", PI * r * r);
    return 0;
}
```

2) macro function

O/P

25

```
# include <stdio.h>
# define SQUARE(x) ((x) * (x))

int main(){
    printf("%d", SQUARE(5));
    return 0;
}
```

3) # undef

Removes a previously defined macro

Ex:

```
# include <stdio.h>
# define VALUE 100
# undef VALUE
```

```
int main(){
#ifdef VALUE
    Printf("Defined");
#else
    printf ("Not Defined");
#endif
    return 0;
}
```

O/P

Not defined

4) Conditional compilation

Allows parts of the code to compile only if certain conditions are true

a) #ifdef

checks whether a macro is defined

```
#include <stdio.h>
#define DEBUG
int main(){
    #ifdef DEBUG
        Printf ("Debug Mode");
    #endif
        return 0;
}
```

O/P

Debug mode

D) #ifndef

checks whether a macro is not defined

```
#include <stdio.h>
int main(){
  #ifndef TEST
    Printf("TEST is not defined");
  #endif
    return 0;
}
```

o/p

TEST is not defined

C) # if

Evaluates a constant expression

```
#include <stdio.h>
#define AGE 18
int main(){
  #if AGE >= 18
    printf("Eligible to vote");
  #endif
    return 0;
}
```

o/p

Eligible to Vote

d) # elif

works like else if

```
#include <stdio.h>
#define MARK 75
int main(){
  #if MARK >= 90
    P ("Grade A");
  # elif MARK >=70
    P(" Grade B");
  # else
    P("G C ");
  # endif
    return 0;
  3.
```

o/p
Grade B

e) # else

Executed if previous conditions are false

```
# if VALVE >100
 P ("High ");
# else
 P ("LOW");
# endif
```

4) #endif

Ends a conditional compilation block

```
#ifdef TEST
  P("Testing");
#endif
```

5) #error

Stops compilation & displays a custom error msg.

Ex:

```
#error Compiler version not supported
```

Compilation Error:

Compiler version not supported.

6) #Pragma

Provides Compiler-specific instruction

Ex:

```
#Pragma once
```

or

```
#Pragma pack(1)
```

used for:-

* structure packing * Disabling compiler warning

4) #endif

Ends a conditional compilation block

```
#ifdef TEST
  P("Testing");
#endif
```

5) #error

Stops compilation & displays a custom error msg.

Ex:

```
#error Compiler version not supported
```

Compilation Error:
Compiler version not supported.

6) #Pragma

Provides compiler specific instruction.

Ex:

```
#Pragma once
    or
#Pragma Pack(1)
```

used for:-

* Structure packing * Disabling compiler warning

* compiler optimization

* preventing multiple inclusion (compiler - specific)

7) # line

changes the compiler's reported line number & filename

Ex:

```
# line 100 "demo.c"
# include <stdio.h>

int main(){
    P("Hello");
    return 0;
}
```

8) # include with Header Guards

Header guards prevent multiple inclusion of the same header file

```
# ifndef MYHEADER_H
# define MYHEADER_H
void display();
# endif
```

File handling in C

File handling in C allows a program to store & retrieve data from files. The C standard library provides file handling functions in the stdio.h header.

Step in file handling

1) Declare a file pointer
2) Open the file
3) Perform file operations (read, write, append, etc
4) close the file

Syntax

```
FILE *fp;
fp = fopen("filename.txt", "mode");
```

File opening modes

Mode	Description
"r"	open for reading. (file must exist)
"w"	open for writing (creates new file or overwrite existing file)
"a"	" " appending (create file if it doesn't exist)

"r+" → Read & write (file must exist)

"w+" → " " " (overwrite existing file)

"a+" → " " append

Function	purpose
fopen()	opens a file
fclose()	closes a file
fprintf()	write formatted data to a file
fscanf()	Reads formatted data from a file
fgetc()	Reads one character
fputc()	writes one "
fgets()	Reads a string (line)
fputs()	write " "
feof()	check end of file
fseek()	moves the file pointer
ftell()	Returns current file pointer Position
Rewind()	moves pointer to beginning of file

Ex 1: writing to a file

o/p (console)

```
#include <stdio.h>
int main(){
    FILE * fp;
    fp = fopen("data.txt", "w");
    if (fp == NULL){
        p("File cannot be opened.");
        return 1;
    }
    fprintf(fp, "Hello, World!");
    fclose(fp);
    printf("Data written successfully.");
    return 0;
}
```

Data written successfully

Contents of data.txt:

Hello, World!

Ex 2: Reading from a file

```
#include <stdio.h>
int main(){
    FILE * fp;
    char str[100];
    fp = fopen("data.txt", "r");
    if (fp == NULL){
        p("File not found.");
```

exist)

isting file)

ose

to a file

from a file

ter Position

eginning of

Ex 1: writing to a file

```
#include <stdio.h>
int main(){
    FILE * fp;
    fp = fopen("data.txt", "w");
    if (fp == NULL){
        P("File cannot be opened");
        return 1;
    }
    fprintf (fp, "Hello, World!");
    fclose (fp);
    Printf ("Data written successfully.");
    return 0;
}
```

o/p (console)

Data written
successfully

Contents of data.txt:

Hello, World!

Ex 2: Reading from a file

```
#include <stdio.h>
int main(){
    FILE * fp;
    char str[100];
    fp = fopen("data.txt", "r");
    if (fp == NULL){
        P("File not found.");
```

```
    return 1;
}
fgets (str, 100, fp);
P("%s", str);
fclose (fp);
return 0;
}
```

o/p
Hello, world!

Ex 3: Appending data

```
# include <Stdio.h>
int main (){
FILE *fp;
fp = fopen ("data.txt", "a");
fprintf (fp, "\n welcome to C Programming.");
fclose (fp);
return 0;
}
```

o/p:
Hello, world!
welcome to C Programming.

Ex 4: Character by character Reading

```
# include <Stdio.h>
int main (){
FILE * fp;
char ch;
fp = fopen ("data.txt", "r");
```

Ex: if (Sensor_fault) {

trigger_alarm();

} else {

~~Hooter off~~

}

Padding

Padding extra (unused) bytes inserted by the compiler b/w structure members to satisfy alignment requirement it improves memory access performance

Ex:

~~int~~ char → 1b → +3 → 4b
int → 4b → 4b
~~short~~ → 2b char → 1b → +3 → 4b

} → It takes as a group of bytes

Packing

Packing is the technique to minimize or remove padding. useful when memory is very limited (e.g. Camera packets, file storage, embedded devices).

Ex:

char c → 1b
int i → 4b
char d → 1b

} → It takes as it is its bytes.

```
while ((ch = fgetc(FP)) != EOF){
    putchar(ch);
}
fclose(FP);
return 0;
}
```

MISRA C (current version in 2023 is C99)

The MISRA C standard is a set of S/W development guidelines for the C Programming langu. created by the motor Industry S/W reliability association (MISRA). why we goes to MISRA C

- ↳ Avoid memory leaks
- ↳ improper variable initialises
- ↳ Standard formats - : C99
- ↳ Security

Ex: `int a=5;`

MISRA

`signed int b=2;`

`unsigned float c=2.5;`

→ we should mention this in the MISRA C

In if else block we should mention it loop runs or not. If any fault occurs in the hardware it should intimate.

Structure →

```
struct test
{
  char c;
  int i;
  char d;
};
```

→ size - 12 → Contains padding to align 'int'. Structure size becomes multiple of 4

Packed structure →

```
#Pragma pack(1)
struct test
{
  char c;
  int i;
  char d;
};
#Pragma pack()
```

→ size - 6 → No padding saves memory use carefully.

union →

```
union Data
{
  int i;
  float f;
  char str[4];
};
```

→ size - 4 → All members share same memory (size of largest member).